Start here: what this code does and how to run it

What this code does

A side-channel attack reads a secret key from power or electromagnetic measurements of a chip. Each measurement is called a **trace**. An attacker trains a model on traces with known keys, then uses it to guess the key behind new traces.

The paper asks a simple question: *how often can an attacker guess the key byte correctly?* It answers with a number called **Bounded Information (BI)**. BI is a guaranteed upper limit on that success rate. It is computed from a plain count: how many of the test traces a model guessed correctly. The count is turned into a limit that holds with high confidence, and the limit is reported in bits.

This repository is the code behind every table and figure in the paper. It has three parts:

- `core/`: the small routines that turn a count of correct guesses into a BI value.
- One folder per experiment: the scripts that run trained attack models on public traces and count their correct guesses.
- `paper_results/`: one script that reruns the experiments and checks each number against the number printed in the paper.

There is one short guide (PDF) per folder. Each guide tells you what the folder shows, what you need before you start, what to type, and what you should see.

Words used in the guides

Word	Meaning here
trace	one measurement of the chip, a list of numbers
attack traces	traces kept aside for testing; the model never saw them
model, attacker	a trained classifier that guesses the key byte from a trace
success count	how many attack traces the model guessed correctly
suite	a fixed list of models that are tested together
certificate, BI	the guaranteed upper limit on the success rate, in bits
archived counts	the success counts saved from the runs behind the paper
recomputed counts	the same counts, produced again on your machine
check	a comparison of a regenerated number with the printed one

Hardware

Nothing here needs a GPU. **Every command in these guides runs on an ordinary CPU.** The outputs shown in the guides were recorded inside Docker on a Linux workstation (x86-64, 12 cores). The same commands were also run inside Docker on a Mac (Apple M3 Pro, 12 cores). Both machines pass every check; the Mac is two to four times slower on most long steps. The tables below list the times on both.

A GPU would only be needed to train the Transformer models again from scratch. That is not needed: the trained models are provided, and the guides only use them to make guesses on traces. Guide 12 shows how to train them again with the optional GPU image, for readers who want to.

The four levels of reproduction

The main script, `paper_results/reproduce.py`, has four levels. Each level recomputes more from scratch, so it needs more downloads and more time.

Level	What it does	Download	Memory	Time on Linux workstation	Time on Mac (Apple M3 Pro)
A	rebuilds all tables and figures from the archived counts and checks them against the paper	none	2 GB	20 seconds	20 seconds
B	recomputes the counts behind Tables 7, 8 and 9 and Figure 2 from the public TCHES20 traces and models	0.5 GB	8 GB	7 minutes	14 minutes
C	recomputes the Transformer rows of Tables 7 and 8 and the BI line of Figure 1 from the provided checkpoints	0.3 GB + 4.4 GB	14 GB	16 minutes for the main part; 46 minutes more for the optional parts	57 minutes for the main part; 2 hours and 24 minutes more for the optional parts
D	recomputes the two results quoted from the full version of the paper	4.4 GB	8 GB	10 minutes, plus the download	26 minutes, plus the download

The four commands:

```
python -m paper_results.reproduce archived      # Level A
python -m paper_results.reproduce tches20       # Level B
python -m paper_results.reproduce estranet      # Level C
python -m paper_results.reproduce fullversion   # Level D
```

The other steps in the guides that take more than a minute:

Step	Guide	Time on Linux workstation	Time on Mac (Apple M3 Pro)
build the Docker image	00	1 minute	2.5 minutes
test the eleven CHES-CTF-2025 Transformer models	03	6 minutes	21 minutes
train the eight byte attackers on raw ASCAD	06	1.5 minutes	14 minutes
combine four bytes over 250 tries	06	3.5 minutes	2.5 minutes
run the artificial-trace sweep	07	1 minute	1 minute
train one scope-1 cell with 50 points	09	3 minutes	4 minutes

The four-byte step is the only one that is faster on the Mac.

Setting up with Docker (recommended)

Docker packs the code and all its libraries into one image, so nothing else has to be installed. Get the code, build the image, and run Level A. The build takes one to three minutes and uses 3.5 GB of disk.

```
git clone https://github.com/K1015/PAC-Certification-of-Multivariate-Side-Channel-Traces.git pac-artifact
cd pac-artifact
docker build -t bi-certification .
```

```
$ docker build -t bi-certification .
[... earlier output omitted ...]
#9 27.43    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
    ━━━━━━━━ 69.6/69.6 kB 438.9 MB/s eta 0:00:00
```

```
#9 27.46 Downloading pyasn1_modules-0.4.2-py3-none-any.whl (181 kB)
#9 27.46    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
    ━━━━━━━ 181.3/181.3 kB 575.3 MB/s eta 0:00:00
#9 27.48 Downloading requests_oauthlib-2.0.0-py2.py3-none-any.whl (24 kB)
#9 27.50 Downloading urllib3-2.8.0-py3-none-any.whl (135 kB)
#9 27.51    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
    ━━━━━━━ 135.7/135.7 kB 261.4 MB/s eta 0:00:00
#9 27.53 Downloading cffi-2.1.1-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (217 kB)
#9 27.53    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
    ━━━━━━━ 217.8/217.8 kB 278.3 MB/s eta 0:00:00
#9 27.56 Downloading oauthlib-4.0.0-py3-none-any.whl (159 kB)
#9 27.56    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
    ━━━━━━━ 159.7/159.7 kB 157.3 MB/s eta 0:00:00
#9 27.58 Downloading pyasn1-0.6.4-py3-none-any.whl (84 kB)
#9 27.58    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
    ━━━━━━━━ 84.4/84.4 kB 671.3 MB/s eta 0:00:00
#9 27.61 Downloading pycparser-3.0-py3-none-any.whl (48 kB)
#9 27.61    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
    ━━━━━━━━ 48.2/48.2 kB 128.0 MB/s eta 0:00:00
#9 28.26 Installing collected packages: mpmath, libclang, flatbuffers, wrapt, urllib3, typing-extensions, te
    rmcolor, tensorflow-io-gcs-filesystem, tensorflow-estimator, tensorboard-data-server, sympy, …
#9 44.06 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour
    with the system package manager. It is recommended to use a virtual environment instead: htt …
#9 44.06 Successfully installed MarkupSafe-3.0.4 absl-py-2.5.0 astunparse-1.6.3 certifi-2026.7.22 cffi-2.1.1
     charset_normalizer-3.5.2 cryptography-50.0.2 filelock-4.0.10 flatbuffers-25.12.19 fsspec-20 …
#9 DONE 45.0s

#10 [6/6] COPY . .
#10 DONE 0.2s

#11 exporting to image
#11 exporting layers
#11 exporting layers 5.6s done
#11 writing image sha256:afd4fa89253ffa07d9dcaaef3ed2e54b4d886b5471a225a735ce060272ad6095 done
#11 naming to docker.io/library/bi-certification done
#11 DONE 5.6s

View build details: docker-desktop://dashboard/build/default/default/alikvdngkzni8tcxpn3dscjie
[exit code 0, 1 min 0 s]
```

Level A is what the image runs when you start it with no command:

```
mkdir -p results
docker run --rm -v "$PWD/results:/artifact/results" bi-certification
```

```
$ mkdir -p results && \
    docker run --rm -v "$PWD/results:/artifact/results" bi-certification
[... earlier output omitted ...]
### Full version: variation across the temperature sweep (bits)

| Benchmark | Raw PI variation | EPI variation | Labels preserved | source |
|---|---|---|---|---|
| ASCAD d0 | 4.23 | 3.05e-10 | yes | archived |
| ASCAD d50 | 23.07 | 2.51e-10 | yes | archived |
| ASCAD d100 | 22.48 | 9.43e-11 | yes | archived |
| AES-HD | 5.30 | 1.78e-10 | yes | archived |
| AES-RD | 27.31 | 7.02e-11 | yes | archived |
| DPAv4 | 20.71 | 8.57e-10 | yes | archived |

[PASS] PI/EPI temperature sweep (full version): 18 values match the paper, 0 differ
[paper_results.fullversion_tables: 0 s, exit code 0]

================ summary ================
Table 7   matches the paper
Table 8   matches the paper
Table 9   matches the paper
Figure 1  matches the paper
Figure 2  matches the paper
Full-version tables matches the paper
```

```
tables and figures: results/paper_results
'created' timestamp seems very low; regarding as unix timestamp
'modified' timestamp seems very low; regarding as unix timestamp
[exit code 0, 20 s]
```

Inside the image, the code sits in the folder `/artifact`. Files written inside a container are lost when it stops, so every folder that receives downloads or results is shared with your machine. The `-v` options do that. The guides use four shared folders. Open a shell inside the container with all four, and type the commands of the other guides at that prompt:

```
mkdir -p results datasets/raw checkpoints external
docker run -it --rm \
  -v "$PWD/results:/artifact/results" \
  -v "$PWD/datasets/raw:/artifact/datasets/raw" \
  -v "$PWD/checkpoints:/artifact/checkpoints" \
  -v "$PWD/external:/artifact/external" \
  bi-certification bash
```

```
$ mkdir -p results datasets/raw checkpoints external && \
    docker run --rm -v "$PWD/results:/artifact/results" -v "$PWD/datasets/raw:/artifact/datasets/raw" -v "$P
    WD/checkpoints:/artifact/checkpoints" -v "$PWD/external:/artifact/external" bi-certification bash -c "pw
    d && \
    ls"
/artifact
Dockerfile
LICENSE
README.md
SHA256SUMS
attacker_scope_diagnostics
bayes_oracle
bi_certification_checkpoints.tar.gz
bi_scope_certificates
bi_suite_8datasets
checkpoints
core
datasets
dimension_stability
docs
epi_comparison
external
locality_diagnostics
model_training
multibyte_composition
paper_results
pyproject.toml
requirements-lock.txt
requirements-ml.txt
requirements.txt
results
scripts
[exit code 0, 0 s]
```

Shared folder	What goes there
`results/`	every table, figure and count file the scripts write
`datasets/raw/`	the downloaded traces
`checkpoints/`	the provided Transformer models (Level C)
`external/`	the Transformer source code (Level C)

Setting up without Docker

You need Python 3.11. Create a virtual environment and install the pinned packages:

```
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install --extra-index-url https://download.pytorch.org/whl/cpu -r requirements-ml.txt
```

The first file is enough for Level A. The second adds TensorFlow and PyTorch, which every script that loads a model needs. Always run the commands from the top folder of the repository.

Downloads

Data	Size	Used by	How to get it
TCHES20 traces and its 480 trained models	0.5 GB (2.3 GB unpacked)	Levels B and D; guides 03, 04, 05, 08, 09, 10	the script `paper_results/fetch_tches20.sh`
Transformer source code (EstraNet)	a few MB	Level C; guides 03, 04	the script `paper_results/fetch_estranet.sh`
Trained Transformer models (162 files)	308 MB	Level C; guides 03, 04	`python -m paper_results.fetch_checkpoints`
ASCAD random-key traces `ASCAD.h5`	420 MB	Level C; guide 03	`python -m paper_results.fetch_ascad_random_key`
CHES-CTF-2025 traces `CHES_Challenge.h5`	4 GB	Level C; guides 03, 07	download by hand into the folder `datasets/raw/ches_ctf_2025/`
Raw ASCAD traces	4.4 GB (6 GB unpacked)	Level D; guide 06	`python -m multibyte_composition.fetch_raw_ascad`

How to read a check

Each table or figure ends with one line like this:

```
[PASS] Table 7 (finite-suite certificates): 80 values match the paper, 0 differ
```

A value matches when it rounds to the number printed in the paper. If a value does not match, the line starts with `[DIFF]` and shows both numbers. When counts are recomputed, you also get a line like `ascad_desync_0: 80/80 recomputed per-model counts equal the archived counts`, which tells you how many of the recomputed counts are the same as the saved ones.